

- [35] Prithviraj Sen, Galileo Namata, Mustafa Bilgic, Lise Getoor, Brian Galligher, and Tina Eliassi-Rad. Collective classification in network data. *AI magazine*, 29(3):93–93, 2008.
- [36] Xujie Si, Mukund Raghothaman, Kihong Heo, and Mayur Naik. Synthesizing datalog programs using numerical relaxation. In Sarit Kraus, editor, *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence, IJCAI 2019, Macao, China, August 10-16, 2019*, pages 6117–6124. ijcai.org, 2019.
- [37] Gustav Sourek, Vojtech Aschenbrenner, Filip Zelezny, Steven Schockaert, and Ondrej Kuzelka. Lifted relational neural networks: Efficient learning of latent relational structures. *Journal of Artificial Intelligence Research*, 62:69–100, 2018.
- [38] Leon Sterling and Ehud Y Shapiro. *The art of Prolog: advanced programming techniques*. MIT press, 1994.
- [39] Charles Sutton and Andrew McCallum. An introduction to conditional random fields for relational learning. *Introduction to statistical relational learning*, 2:93–128, 2006.
- [40] Efthymia Tsamoura and Loizos Michael. Neural-symbolic integration: A compositional perspective. *CoRR*, abs/2010.11926, 2020.
- [41] Emile van Krieken, Erman Acar, and Frank van Harmelen. Analyzing differentiable fuzzy implications. In Diego Calvanese, Esra Erdem, and Michael Thielscher, editors, *Proceedings of the 17th International Conference on Principles of Knowledge Representation and Reasoning, KR 2020, Rhodes, Greece, September 12-18, 2020*, pages 893–903, 2020.
- [42] Jason Weston, Frédéric Ratle, Hossein Mobahi, and Ronan Collobert. Deep learning via semi-supervised embedding. In *Neural networks: Tricks of the trade*, pages 639–655. Springer, 2012.
- [43] Zhun Yang, Adam Ishay, and Joohyung Lee. Neurasp: Embracing neural networks into answer set programming. In *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence, IJCAI*, pages 1755–1762, 2020.
- [44] Xiaojin Zhu, Zoubin Ghahramani, and John D Lafferty. Semi-supervised learning using gaussian fields and harmonic functions. In *Proceedings of the 20th International conference on Machine learning (ICML-03)*, pages 912–919, 2003.

A Appendix

A.1 Data and Licenses

- For tasks T1, T3 and T4, we used the MNIST dataset from [20] to generate new datasets. The MNIST dataset itself was released under the Creative Commons Attribution-Share Alike 3.0 license.
- The Handwritten Formula Recognition (HWF) dataset (used in T2) originates from [21].
- The Cora and Citeseer datasets (T5) are from [35].
- The dataset of the Word Algebra Problem (T6) originates from [32].

A.2 Computational details

Inference time experiments are all executed on a MacBookPro 13 2020 (2.3 GHz Quad-Core Intel Core i7 and 16 GB 3733 MHz LPDDR4).

B Task Details

In this section we show the full DeepStochLog programs and additional experimental details for all tasks.

B.1 MNIST Digit Addition

For the MNIST addition problem, we trained the model for 25 epochs using the Adam optimizer with a learning rate of 0.001 and used 32 training terms in each batch for each digit length. DeepStochLog program for single-digit numbers:

```
1 digit(Y) :- member(Y,[0,1,2,3,4,5,6,7,8,9]).
2 nn(number, [X],[Y],[digit]):- number(Y) --> [X].
3 addition(N) --> number(N1), number(N2), {N is N1+N2}.
```

DeepStochLog program for L -long digits numbers:

```
1 digit(Y) :- member(Y,[0,1,2,3,4,5,6,7,8,9]).
2 nn(number, [X],[Y],[digit]):- number(Y) --> [X].
3 addition(N) --> number(N1), number(N2), {N is N1+N2}.
4 0.5::multi_addition(N, 1) --> addition(N).
5 0.5::multi_addition(N, L) --> {L > 1, L2 is L - 1},
6                               addition(N1), multi_addition(N2, L2),
7                               {N is N1*(10*L2) + N2}.
```

B.2 Handwritten Mathematical Expressions

For the Handwritten Mathematical Expression problem, we trained the model for 20 epochs using the Adam optimizer with a learning rate of 0.003 and a batch size of 2. We used two separate, similar neural networks for recognising the numbers and the operators (see Table 8. The encoder of the neural networks has a convolutional layer with 1 input channel, 6 output channels, kernel size 3, stride 1, padding 1, a ReLu, max pooling with kernel size 2, a convolutional layer with 6 inputs, 16 outputs, kernel size 3, stride 1, padding 1, a ReLu, a max pooling with kernel size 2, and a 2d dropout layer with $p = 0.4$. They then use two fully connected layers, one from 1936 to 128 with a ReLu, and one linear to the number of classes (10 and 4 respectively). This problem is modeled as follows in DeepStochLog:

```

1 digit(Y) :- member(Y,[0,1,2,3,4,5,6,7,8,9]).
2 operator_d(Y) :- member(Y,[plus, minus, times, div]).
3 term_switch_d(Y) :- member(Y,[0, 1, 2]).
4 e_switch_d(Y) :- member(Y,[0, 1, 2]).
5
6 nn(number, [X],[Y],[digit]):: number(Y) --> [X].
7 nn(operator, [X],[Y],[operator_d]) :: operator(Y) --> [X].
8 1 :: factor(N) --> number(N).
9
10 nn(term, [], [Y], [term_switch_d]) :: term(N) --> term_switch(N,Y).
11 0.33 :: term_switch(N, 0) --> factor(N).
12 0.33 :: term_switch(N, 1) --> term(N1), operator(times), factor(N2),
13     {N is N1 * N2}.
14 0.33 :: term_switch(N, 2) --> term(N1), operator(div), factor(N2),
15     {N2>0, N is N1 / N2}.
16
17 nn(expression, [], [Y], [e_switch_d]) :: expression(N) --> e_switch(N,Y).
18 0.33 :: e_switch(N,0) --> term(N).
19 0.33 :: e_switch(N,1) --> expression(N1), operator(plus), term(N2),
20     {N is N1 + N2}.
21 0.33 :: e_switch(N,2) --> expression(N1), operator(minus), term(N2),
22     {N is N1 - N2}.

```

B.3 Well-formed Parentheses

We ran the well-formed parenthesis problem for 1 epoch using the Adam optimizer with a learning rate of 0.001 and a batch size of 4. This problem is modelled as follows:

```

1 bracket_d(Y) :- member(Y,["(",")"]).
2 s_switch_d(Y) :- member(Y,[0,1,2]).
3
4 nn(bracket_nn,[X], [Y], [bracket_d])::bracket(Y) --> [X].
5 nn(s_nn,[X],[Y],[s_switch_d])::s --> s_switch(Y).
6 0.33::s_switch(0) --> s, s.
7 0.33::s_switch(1) --> bracket("("), s, bracket(")").
8 0.33::s_switch(2) --> bracket("("), bracket(")").

```

B.4 Context-Sensitive Grammar

The model was trained on the canonical context-sensitive grammar $a^n b^n c^n$ problem for 1 epoch using the Adam optimizer with a learning rate of 0.001 and batch size 4. This problem is modelled in DeepStochLog as follows:

```

1 rep_d(Y):- domain(Y, [a,b,c]).
2 0.5:: s(0) --> akblcm(K,L,M),{K=L; L=M; M=K}.
3 0.5:: s(1) --> akblcm(N,N,N).
4 akblcm(K,L,M) --> rep(K,A), rep(L,B), rep(M,C),{A=B, B=C, C=A}.
5 0.5 :: rep(s(0), C) --> terminal(C).
6 0.5 :: rep(s(N), C) --> terminal(C), rep(N,C).
7 nn(mnist, [X], [C], [rep_d]) :: terminal(C) --> [X].

```

B.5 Semi-supervised classification in citation networks

The model was trained on the Cora and Citeseer datasets using the Adam optimizer with a learning rate of 0.01. We trained for 100 epochs and we selected the model corresponding to the epoch with